Intermediate (make search conditional) — Solution

> Back to the exercise · Hint 1 · Hint 2 · Solution

Story — `search_tool_integration_practice.py`: this is the exact conditional edge the Build Task needs at its very first step — decide, before anything else runs, whether this question needs the documents at all. **If not:** the Build Task would search on every question, paying for an embedding call and padding the prompt even for "what is 2 + 2?".

Uses the same `practice/` setup as the Basic exercise (Doc08's files, `docs/` and `kb.py`).

Solution

```
# search_tool_integration_practice.py
from typing import Callable, TypedDict

from langchain_openai import ChatOpenAI
from langgraph.graph import END, START, StateGraph

from kb import collection
from retriever import retrieve

model = ChatOpenAI(model="gpt-4o-mini", temperature=0)
KEYWORDS = ["coffee", "brew", "grind", "water", "beans", "according to"]

class QAState(TypedDict):
    task: str
    sources: list[dict]
    answer: str

def search_node(state: QAState) -> dict:
    return {"sources": retrieve(collection, state["task"])}

def answer_node(state: QAState) -> dict:
    if not state["sources"]:
        return {"answer": model.invoke(state["task"]).content}
    pieces: list[str] = []
    for source in state["sources"]:
```

```
        pieces.append(source["text"])
    context = "\n\n".join(pieces)
    prompt = f"Context:\n{context}\n\nQuestion: {state['task']}"
    return {"answer": model.invoke(prompt).content}

def should_search(state: QAState) -> str:
    task = state["task"].lower()
    has_match = False
    for keyword in KEYWORDS:
        if keyword in task:
            has_match = True
    if has_match:
        return "search"
    return "skip"

def build_graph(router: Callable[[QAState], str]):
    builder = StateGraph(QAState)
    builder.add_node("search_node", search_node)
    builder.add_node("answer_node", answer_node)
    builder.add_conditional_edges(
        START, router, {"search": "search_node", "skip": "answer_node"}
    )
    builder.add_edge("search_node", "answer_node")
    builder.add_edge("answer_node", END)
    return builder.compile()

def run(graph, task: str) -> dict:
    return graph.invoke({"task": task, "sources": [], "answer": ""})

if __name__ == "__main__":
    graph = build_graph(should_search)
    for task in ["How hot should the water be when brewing?", "What is 2 + 2?"]:
        result = run(graph, task)
        print(f"{task!r} -> searched: {len(result['sources']) > 0}")
```

Expected output:

```
'How hot should the water be when brewing?' -> searched: True
'What is 2 + 2?' -> searched: False
```